AGENTIC SOFTWARE ARCHITECTURE · PART 1 · CLASS 1

Agent anatomy, reliability and the token cost model

Sun 8 Nov 2026 · Part 1 · core class

What you will be able to do

- Name the parts of an **agent** and tell a **workflow** from an agent
- Read a **ReAct trace** line by line
- Explain why n steps at success p give **p^n**, and why **pass^k** matters
- Say why the same prompt gives **different answers**
- Explain **BPE**, byte fallback and why **Bangla** costs more
- Estimate the **cost** of a request before building the feature

Today’s topics

#	Topic
1	Course map: one repo, two agents
2	Recap: tokens and context
3	Agent anatomy, workflow vs agent, ReAct
4	Quality attributes, p^n, pass^k
5	The LM as probabilities
6	BPE, byte fallback, Bangla fertility, cost model
7	Lab and the lab rule

Course map: one repo, two agents

Part 1 · 8–22 Nov
The Model as a Measured Component

Part 2 · 29 Nov–13 Dec
The Agent as Tested Subsystems

Part 3 · 20 Dec–3 Jan
The Harness as a Control Layer

Part 4 · 10–24 Jan
The Architecture as an Evaluated Claim

- 12 Sunday classes, 4 parts of 3, one instructor and one mini-deliverable per part
- **Two agents:** the one you build, and the coding agent that helps write it under rules
- Part 1 deliverable: tested 3-tool agent, tag **part1-ref**

Class 1 · Agent anatomy, reliability and the token cost model

What you carry from class to class

Class	The repo gains	The ledger gains
1	LM wrapper: cache, ledger, budget cap	Bangla/English cost ratio
2	Versioned schema, contract test	Invalid rate, field accuracy, cost
3	Agent loop, scripted tests, 20-task eval	pass^3 per task
4–12	Retrieval, MCP, memory, harness, paper	One number per class

Results ledger: one number per lab, kept all course

Recap: shared words

Token
the unit the model reads and writes

Context window
the most tokens one call can hold

Prompt
everything we send in

Completion
what the model sends back

Source: Stanford CS329Z L1–L2 slides (tokens and context)

Agent = model + tools + loop + memory

Model

next-token predictor; has no memory

Tools

plain functions our code runs

Loop

step cap and stop rule

Memory

history re-sent each step

...wrapped in code that decides when to stop

Workflow vs agent

	Workflow	Agent
Who picks the next step?	Our code	The model, each turn
Predictable?	High	Lower
Cost and latency	Known in advance	Vary per run
Example	extract → validate → save	model picks search, read, compute

Poll: “always translate, summarise, email” = workflow · “decides to search, calculate or ask” = agent

ReAct: the loop

Thought
model reasons

Action
tool call, as JSON

Observation
tool result back in

Repeat until **final answer** or **step cap**

A ReAct trace, line by line

```
Task: days until the tuition fee deadline? today 2026-11-22
Assistant: {"tool":"search_notices","args":{"query":"tuition"}}
Observation: [{"id":2,"title":"Tuition fee payment"}]
Assistant: {"tool":"get_notice","args":{"id":2}}
Observation: {"id":2,"deadline":"2026-11-30"}
Assistant: {"tool":"days_until", ...} Observation: 8
Assistant: {"final":"8 days"}
```

- Assistant lines = model output · Observation lines = **our code**
- Every line costs tokens; the whole history is re-sent each step
- 4 model calls = 4 chances to fail

Quality attributes of an agent

Attribute	Question	Measured by
Cost	What does a task cost?	tokens × price (ledger)
Latency	How long does it take?	wall-clock, time to first token
Reliability	How often does it succeed?	p^n and pass^k
Security	Can it be made to do harm?	probe tests (Part 3)

A quality attribute is a measurable property that is not a feature

Quality attribute scenario

Source
who or what triggers

Stimulus
the event

Artifact
the part affected

Environment
conditions at the time

Response
what the system does

Response measure
the number we test

A scenario built from p^n

Part	Example
Source	a student
Stimulus	asks a 4-step question
Artifact	the notice agent
Environment	normal operation
Response	returns the correct answer
Response measure	pass^3 ≥ 0.9 and under $0.001 per task

Pair task: write one scenario for latency

The reliability gap

0.60

0.95 per step × 10 steps

chain success = p^n

0.99 per step → 0.90

Good steps do not make a reliable chain.

Task success = p^n

p per step	n = 1	n = 5	n = 10	n = 20
0.99	0.990	0.951	0.904	0.818
0.95	0.950	0.774	0.599	0.358
0.90	0.900	0.590	0.349	0.122

Your turn: compute 0.95^10 on paper

Read p^n as a warning

Assumes independence

and that one failure ruins the task

Agents can recover

errors return as observations

Success ≠ safety

99% task success can still break a rule

So: measure reliability, do not assume it

pass@k vs pass^k

Metric	Meaning	Reads as
pass@1	average single attempt	typical
pass@k	at least one of k succeeds	optimistic
pass^k	all k succeed	reliable

Our headline: pass^3 on 20 tasks

The model is probabilities

Logits | Softmax | **Temperature** | Sampling | Next token

- Low temperature: predictable
- High temperature: varied
- Temperature 0 is not a guarantee of identical output

Worked example: scores 2.0 / 1.0 / 0.1

Temp.	P(A)	P(B)	P(C)	log P(A)	log P(B)	log P(C)
1.0	0.659	0.242	0.099	−0.417	−1.417	−2.317
0.5	0.864	0.117	0.019	−0.146	−2.146	−3.946
2.0	0.502	0.304	0.194	−0.690	−1.190	−1.640

- Lower temperature → the top token takes more probability
- Higher temperature → flatter, more varied

Why the same prompt differs

- Temperature > 0: the pick is **random**, weighted by the probabilities
- Temperature 0: always the largest. Mostly repeatable, **not guaranteed**
- **Log-probability** shows confidence in logs
- P(sequence) = product of token probabilities, so log P = **sum** of log-probs

Want repeatable extraction in Class 2? Choose temperature 0.

Why tokens at all

Letters

too fine: very long sequences

Subwords

the compromise

Words

too coarse: millions of rare words

Cost, speed and context limits are all counted in tokens

BPE: the merge algorithm

1. Start from **bytes**
2. Count adjacent pairs
3. **Merge** the most frequent pair into a new token
4. Repeat for a fixed number of merges

`t h e _ c a t` → `th e _ c a t` → `the _ c at` → `the _ cat`

Toy run: 740 bytes → 163 tokens after 12 merges

Byte fallback and Bangla

hello

5 characters · **5 bytes**

common word: often 1 token

বাংলা

5 characters · **15 bytes**

few merges known: up to 15 byte tokens

- UTF-8: 1 byte per English letter, **3 bytes per Bangla letter**
- **Byte fallback:** unknown text falls back toward bytes

Fertility vs cost ratio

Fertility

tokens per **word**

Cost ratio

Bangla tokens ÷ English tokens on **aligned pairs** (same meaning)

- Use the **cost ratio** for money: words do not line up across languages
- Sample pair: 5 English words, 3 Bangla words
- Worst case (one token per byte) on the 5-pair sample: **2.85**. Real tokenizers will be lower

Cost model

ONE CALL

$$\text{cost} = \text{input} \times \text{price_in} + \text{output} \times \text{price_out}$$

N-STEP AGENT

$$\text{total input} = n \cdot B + h \cdot n(n-1)/2$$

History is re-sent: twice the steps costs more than twice

Same meaning, 2.85× the bill

Request	Prompt	Completion	Cost
English	500	200	$0.000195
Bangla, ratio 2.85	1,425	570	$0.000556

- Notional prices: $0.15 input, $0.60 output per million tokens
- Also **slower**: more tokens to process and to generate
- In an agent loop the extra tokens are paid again at every step

History is re-sent every step

Steps n	Input tokens (B = 300, h = 120)
1	300
3	1,260
6	3,600
12	11,520

total input = n·B + h·n(n−1)/2 → twice the steps, more than 3× the cost

Lab: what you build today

Step	Task
Setup	Install Ollama or llama.cpp and a 3B model (LAN server fallback)
Build	Write the LM wrapper: reply cache, cost ledger, budget cap
Measure	Bangla vs English tokens on three tokenizers
Check	Pass check, results ledger, verification log

Laptop with 8 GB RAM; local models first, so no API keys needed

The LM wrapper: three features

Reply cache

key = model + prompt + every parameter + seed

Cost ledger

one JSON line per call: tokens, cost, cached or not

Budget cap

checked **before** the call; over the cap is refused

One test per feature · a fake backend makes them free and repeatable

The lab rule, for the whole course

A coding agent may write code only after you write the test.

1 · Red
write the test, watch it fail

2 · Green
smallest code that passes

3 · Refactor
tidy while tests stay green

Log three lines per lab: what it wrote · which test proves it · what you reviewed

Measure Bangla vs English

1. Load the **200 Bangla–English** sentence pairs
2. Tokenize both sides with each tokenizer
3. Per pair: Bangla tokens ÷ English tokens
4. Report the ratio with its **spread**, not one number

Qwen2.5	BLOOM	o200k_base

Ledger number today: the Bangla/English cost ratio

Pass check, in the room

Cache hit
a repeated call does not reach the model

Cap refusal
a call over the cap is refused

Cost-ratio table
Bangla/English ratio for three tokenizers

Verification log
first entry: three lines

All four must show before you leave

Three rules for all of Part 1

1. **Test first.** See it fail.
2. **One number** per lab in the results ledger.
3. **Spread, not one run.** pass^k, not pass@k.

Next week: KV cache · structured output · schema contract

Sources and pre-reading

- Stanford CS329Z L1: Introduction, what are agentic systems?
- Stanford CS329Z L2: LLMs for builders
- Zaharia et al., *Compound AI Systems* (BAIR 2024), pre-reading
- Ollama docs · Hugging Face tokenizers docs

Research pathway: Bangla fertility is a publishable measurement